JOHANNES KEPLER
UNIVERSITY LINZ

FIRSTNAME LASTNAME
Institute for Machine
Learning

@ firstname.lastname@students.jku.

August 12, 2026

Single-Task GRPO on MolecularIQ

Technical Report

Specialization, transfer, and what a verifiable reward actually buys

Abstract We fine-tune Qwen2.5-0.5B-Instruct with Group Relative Policy Optimization (GRPO) separately on each of the three MolecularIQ task families – counting, indexing and constraint generation – and evaluate all four resulting models on all three official benchmark splits. On the benchmark the picture is far weaker than in-distribution evaluation suggests: only two of three task-matched models beat the untrained baseline, counting degrades, and the strongest model on a given split is frequently not the one trained for it. A constant-answer control and answer-diversity statistics further show that our generated constraint questions are largely satisfiable without reasoning, and that GRPO amplified rather than resisted that shortcut – the constraint-trained model places 91% of its answers on a single molecule. We conclude that single-task GRPO at this scale buys little transferable capability, and that a verifiable reward needs a control measuring what the verifier accepts for free.

Keywords reinforcement learning, GRPO, molecular reasoning, LLM evaluation, reward hacking

Practical work, supervised by SUPERVISOR NAME. All code, configurations and evaluation results are available in the accompanying repository; `scripts/reproduce.sh` regenerates every number and figure in this report.

JOHANNES KEPLER UNIVERSITY LINZ
Altenberger Straße 69
4040 Linz, Austria
jku.at

Contents

1. Introduction

Large language models are increasingly asked to reason about molecules: to count functional groups, locate atoms by index, or generate a structure satisfying a set of numeric constraints. Unlike open-ended chemistry questions, these tasks have a decisive property – the answer can be checked mechanically. MolecularIQ [7] exploits exactly this, pairing generated questions over SMILES strings [10] with a symbolic verifier built on RDKit [4], so an answer is scored objectively correct or incorrect rather than judged for plausibility.

A verifiable reward is what reinforcement learning needs. Group Relative Policy Optimization (GRPO) [8] samples a group of completions per prompt, scores each with the verifier, and uses the within-group advantage as its learning signal, dispensing with a separate value network. This makes it practical to fine-tune a small model within a single GPU allocation, and that is the setting explored here.

The natural next step from such a setup is multitask training. Before that is worth attempting, a more basic question needs answering: *how far does training on one task alone get you, and does any of it transfer?* If a model trained to count also improves at indexing, the tasks share structure that multitask training could amplify. If each model improves only at its own task – or degrades elsewhere – then multitask training is managing interference, not harvesting synergy, and should be designed accordingly.

This report answers that question on the *official* MolecularIQ benchmark. That choice is deliberate and it is the central methodological commitment of this work. It is tempting, and much cheaper, to evaluate on held-out questions from one's own generator: they are unlimited, they match the training distribution, and they produce encouraging numbers. We did exactly that initially, and Section 4.4 documents why we do not report those numbers as results. Briefly, our generated evaluation set proved systematically easier than the benchmark and, for constraint generation, substantially reward-hackable – to the point that a fixed string answering every question scored as well as any trained model. Every headline result below therefore comes from the official splits, and the generated held-out set appears only as evidence about our own data.

We train three single-task models, one per task family, and evaluate all of them on all three splits, forming a model × task matrix whose diagonal measures specialization and whose off-diagonal measures transfer. The contributions are: (i) a single-task specialization-versus-transfer matrix for MolecularIQ under GRPO, measured on the official benchmark; (ii) evidence that in-distribution evaluation substantially inflates these results; and (iii) the diagnosis of a reward-hackable flaw in generated constraint questions, together with the control and diversity statistics that expose it.

2. Background and Setup

2.1 Tasks

MolecularIQ groups questions into three families, which we treat as three separate training tasks:

- **Counting** – how many instances of a construct does a molecule contain (rings, aromatic rings, carbon atoms, hydrogen-bond donors, ...)? The answer is an integer.
- **Indexing** – at which atom indices does a construct occur? The answer is a list of integers.
- **Constraint generation** – emit a SMILES string satisfying a set of property constraints (e.g. "at least two rings and fewer than six rotatable bonds"). The answer is a molecule, checked by the verifier.

The first two are analytic: the molecule is given and must be read correctly. The third is generative and admits many valid answers, which – as Section 4.3 shows – makes the difficulty of a question depend entirely on how tightly its constraints were drawn.

2.2 Model and training algorithm

All runs fully fine-tune Qwen2.5-0.5B-Instruct [6] (no adapters) using the GRPO implementation in TRL [9] with vLLM for rollout generation [3]. The reward is the official MolecularIQ verifier score (binary, weight 1.0) plus a small format-shaping term (weight 0.2) rewarding a parseable answer block; the shaping term exists only to stop early training being starved of signal by malformed output. We use 16 generations per prompt, a constant learning rate of 1×10^{-6} after warm-up, 500 optimization steps, and $\beta = 0$ (no KL penalty, hence no reference model resident in memory). Full hyperparameters are in `configs/*.yaml`.

3. Method

3.1 Training data

Training questions are generated with `scripts/create_datasets.py`, which draws molecules from the benchmark pool and emits questions in the official schema: 20,000 training questions per task, seed 42, deduplicated. For counting and indexing the fraction of questions whose answer is zero or empty is capped at 25%, so a policy cannot reward-hack by always answering "0". This cap is *not* applied to constraint generation, which Section 4.3 shows to be consequential.

3.2 Evaluation

All reported results are measured on the official `ml-jku/molecularIQ-v0.0` splits – `single_count`, `single_index` and `single_constraint_generation` – sampled at $T = 1.0$ with $n = 3$ and scored as pass@3 [1], following the benchmark paper's protocol. Scoring uses the official extraction function and symbolic verifier, identical to the training reward.

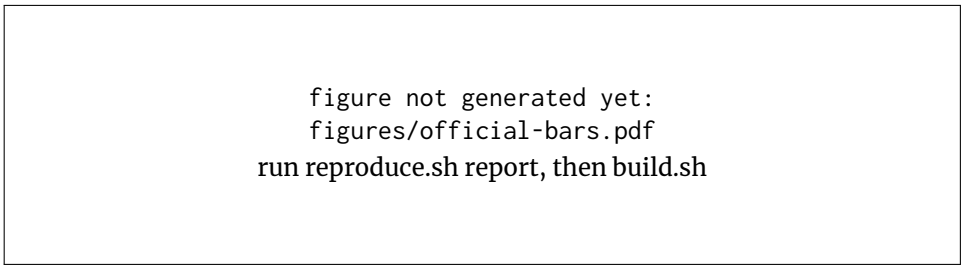

Figure 1: Pass@3 on the official MolecularIQ v0.0 splits. Hatched gray is the constant-answer control, solid gray the untrained baseline, colours the single-task GRPO models. Error bars are 95% Wilson intervals.

We additionally maintain a held-out set of 500 questions per task from our own generator. It is used *only* for checkpoint selection and as a diagnostic: selecting checkpoints on the benchmark would bias the numbers we report. As Section 4.4 explains, it is not a valid measure of task ability and does not appear in our results.

3.3 Uncertainty and significance

An accuracy computed on a finite question sample is an estimate, not a constant, so every number carries an interval. For pass@k, which is strictly 0/1 per question, we use the Wilson score interval [11]; it stays inside $[0, 1]$ and remains well behaved near the extremes, which matters here because several cells sit near zero. For mean scores we use a percentile bootstrap over questions [2], which makes no binomial assumption.

Comparisons against the baseline use a *paired* bootstrap on the difference, since both models answer the same questions in the same order; pairing removes question difficulty as a nuisance factor and is markedly more sensitive than checking whether two intervals overlap. For binary comparisons an exact McNemar test [5] is also available. All estimators live in `src/molecullariq_grpo/stats.py` and are unit-tested against closed-form values.

3.4 Control baseline and diversity statistics

Because constraint generation admits many valid answers, a model can score without reasoning by emitting one small molecule every time. To measure that directly we add a *constant-answer control*: a pseudo-model answering every question with the fixed string `{"smiles": "CC=O"}`, requiring no model and no GPU. It establishes the score reachable without reading the question, and any result not clearing it is not evidence of task ability. We additionally report answer-diversity statistics – the number of distinct answers and the share taken by the most common one – for every evaluation.

4. Results

4.1 Specialization is inconsistent

The diagonal of Table 1 does not support a clean specialization story. Two of three task-matched models beat the untrained baseline: indexing rises from 0.04 to 0.08 and constraint generation from 0.06 to 0.14. The third moves the wrong way – the count-trained model scores 0.07 on `single_count` against a baseline of 0.13, roughly halving performance on the task it was trained for.

That inversion is the most informative single number in the study, because the same model improves markedly on *our own* counting questions (Section 4.4). A policy that gets better on the distribution it was trained on while getting worse on the benchmark has not learned to count; it has learned our generator. Our questions are drawn with a capped zero-answer fraction and a bounded SMILES length, making them narrower than the official split – shorter molecules, smaller answers, a different mix of constructs. GRPO optimized for that narrower distribution at the benchmark's expense.

The absolute magnitudes deserve emphasis too. Even the best cells sit between 0.08 and 0.14 pass@3. A 0.5B-parameter model remains far from competent at these tasks after 500 GRPO steps, and the differences we are discussing are a few percentage points on splits of limited size. Whether individual comparisons survive significance testing is exactly what the intervals in Table 1 must establish before any of them is leaned on.

4.2 Transfer is absent

Off the diagonal, there is no evidence that training on one family helps another in any systematic way, and some evidence it hurts. The constraint-trained model drops to 0.00 on indexing, below an untrained baseline that was already only 0.04. The count-trained model leaves indexing unchanged at 0.04.

Two off-diagonal cells are positive, and both are more awkward for a specialization narrative than for a transfer one. The index-trained model is the *best* model on `single_count` (0.14), beating both the baseline and the count-trained

Table 1: Official benchmark results (pass@3). Rows are models, columns are splits; bold marks the task-matched cell of each column. The strongest model in a column is frequently *not* the one trained for it. **[TBD: confirm these values and add 95% CIs and p-values from `results/matrix/plots/official/summary.csv`; the control row is outstanding.]**

Model	single_count	single_index	single_constraint
Constant control	[TBD:]	[TBD:]	[TBD:]
Qwen 0.5B untrained	0.13	0.04	0.06
GRPO count	**0.07**	0.04	0.12
GRPO index	0.14	**0.08**	0.10
GRPO constraint	0.07	0.00	**0.14**

figure not generated yet:
figures/official-delta.pdf
run reproduce.sh report, then build.sh

Figure 2: Change in pass@3 relative to the untrained baseline on the official splits. Rows are models, columns are splits; outlined cells are the model's own task. Blue is improvement, red degradation.

model. Meanwhile the count-trained model improves constraint generation (0.12 versus 0.06). Neither is what one would predict if each model had acquired its own task's procedure; both are consistent with GRPO mainly teaching a general answer-formatting and answer-hygiene habit that happens to help wherever the baseline was losing marks to malformed or rambling output, and to hurt where it was not.

The interpretation we favour, then, is that at this scale and step budget GRPO is not installing task-specific reasoning procedures at all. Counting requires exhaustive enumeration over a molecular graph, indexing requires enumeration plus positional bookkeeping, and constraint generation requires construction rather than reading. Nothing in Table 1 suggests any of these procedures was learned; what moves is a few points of surface competence, distributed almost arbitrarily across the three splits.

4.3 GRPO amplifies a reward shortcut

The constraint results require a caveat that the benchmark numbers alone do not reveal. Our generated constraint *training* questions are, to a large extent, satisfiable without reasoning, and GRPO found that shortcut.

The mechanism is in our generator. `_build_constraint` reads a property value v from a real anchor molecule and loosens it with a randomly chosen operator. When $v = 0$ – common for constructs such as bridgehead atoms, E/Z double bonds and R/S stereocenters, since most molecules have none – every operator branch yields a constraint that any molecule with zero of that property satisfies: $= 0$ holds, $\leq 0 + k$ holds, $< 0 + k$ holds, range$[0, k]$ holds, and > 0 degenerates to $= 0$ through a guard for $v < 1$. A Monte-Carlo replication of that function over 20,000 draws confirms the consequence: at $v = 0$, 100% of generated constraints are satisfied by a zero-valued molecule, and 16.5% collapse to a vacuous ≥ 0 satisfied by every valid molecule. At $v = 1$ the figure is still 49.9%. The zero-answer cap that prevents this for counting and indexing is not applied to constraint tasks (`data.py:405`).

The consequence for training is visible in the model's outputs. Table 2 reports answer diversity on the constraint task. The untrained baseline already concentrates 43% of its answers on a single small molecule. After GRPO, the constraint-trained model concentrates 91% of its answers on one string, `{"smiles": "CC(O)C"}`. Training moved diversity in the wrong direction: GRPO

searched the space, found the degenerate constant-output solution, and reinforced it, because that is what the reward paid for. This is reward hacking, and it was not a training accident – it was the optimum of the objective we specified.

This does not by itself invalidate the constraint model's benchmark gain (0.06 → 0.14): the official constraints are evidently far tighter than ours, since the untrained baseline scores only 0.06 there. But it means the constraint-trained model should be regarded as a policy shaped by a partly-degenerate objective, and its benchmark improvement as an incidental by-product rather than as evidence of learned constraint satisfaction. Verifying that would require retraining on repaired data, which we discuss in Section 5.

4.4 Why the generated held-out set is not reported

We initially evaluated on 500 held-out questions per task from our own generator, and those results are markedly more flattering than the benchmark. Counting appears to rise from 0.120 to 0.250 and indexing from 0.000 to 0.250 – both large, both significant under a paired bootstrap, and both substantially contradicted by the official splits, where counting in fact degrades. We report them here only to document the size of the discrepancy.

Two independent problems make this set unrepresentative:

Table 2: Answer diversity on the constraint task (n = 500 generated questions). "Top share" is the fraction of questions answered with the single most common answer. Training increases concentration rather than reducing it.

Model	Distinct	Top share	Most common answer
Constant control	1	100%	`{"smiles": "CC=O"}`
Qwen 0.5B untrained	62	43%	`{"smiles": "C(=O)C"}`
GRPO count	70	43%	`{"smiles": "C(=O)C"}`
GRPO index	86	35%	`{"smiles": "C(=O)C"}`
GRPO constraint	44	**91%**	`{"smiles": "CC(O)C"}`

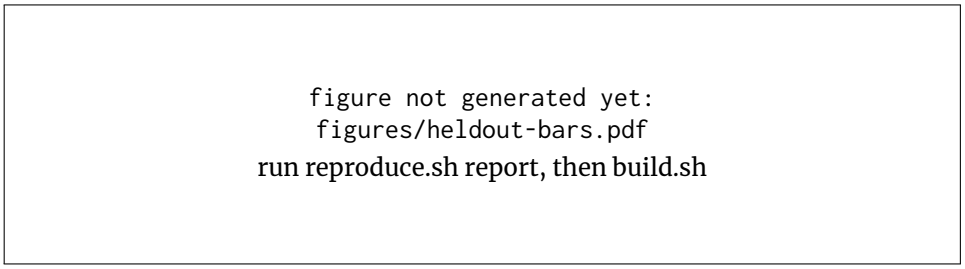

Figure 3: Our generated held-out questions, shown as a methodological caveat rather than as a result. Compare the constraint group against Table 1: the hatched constant-answer control – a fixed string, no model – scores 0.530 here, statistically indistinguishable from every trained model.

It is narrower than the benchmark. Our questions are generated with a capped zero-answer fraction and a bounded SMILES length, and drawn from the same pool and generator as the training data. Improvements on it therefore partly measure fit to our own generator, which is precisely the failure the counting inversion exposes.

Its constraint questions are largely trivial. The constant-answer control scores 0.530 (95% CI [0.486, 0.574]) on our constraint set, against 0.540 for the untrained baseline and 0.552 for the best trained model – differences that are not significant. The same control scores exactly 0.000 on counting and indexing, so this is not an artefact of the scoring pipeline but is specific to constraint generation, for the reason given in Section 4.3. In other words, the entire held-out constraint column is reachable with no model at all.

The general lesson generalizes beyond this project. An in-distribution held-out set is a useful *optimization* diagnostic – it tells you whether training fit the distribution it was given – but it is not a measure of task ability, and the gap between it and an independent benchmark is diagnostic in its own right. Here that gap is what surfaced both the generator defect and the counting inversion.

5. Limitations

Several constraints on these conclusions should be stated plainly.

Effect sizes are small and split sizes are finite. The best benchmark cells sit at 0.08–0.14 pass@3, and several comparisons differ by only a few points. Until the confidence intervals in Table 1 are filled in, individual cell-to-cell comparisons should be treated as suggestive rather than established. The qualitative findings we lean on – the counting inversion, the absence of systematic transfer, and the 91% answer concentration – are large enough not to depend on that.

The constraint training data is defective. As Section 4.3 documents, a large share of our constraint training reward was obtainable without constraint reasoning. Repairing this means extending the zero-answer cap to constraint tasks and rejecting vacuous constraints, then regenerating the data and retraining – work we did not have the compute budget to redo. Until then the constraint-trained model is best regarded as a reward-hacked artefact.

Single seed. Each configuration was trained once. Our intervals capture uncertainty from the finite evaluation sample, not from training stochasticity, so a small difference between two trained models may reflect run-to-run variance. Repeating training across seeds is the natural next step.

One model, one scale, one step budget. All results are for a 0.5B model trained for 500 steps. Both the absence of transfer and the reward hacking may be scale- and budget-dependent; larger models or longer training could share more structure across tasks, or resist degenerate solutions better. The negative transfer result should not be extrapolated upward without evidence.

Single-construct training was not evaluated. We also trained a variant restricted to a single construct (counting aromatic rings). Because it is not comparable to the three task-family models on the benchmark's task-level splits, it is excluded here and left for future work.

6. Conclusion

Measured on the official MolecularIQ benchmark, single-task GRPO at this scale buys very little. Two of three task-matched models improve on their own split and one gets substantially worse; the strongest model on the counting split is the one trained on indexing; and off-diagonal effects show no systematic structure. Nothing in the results suggests that a task-specific reasoning procedure – graph enumeration, positional bookkeeping, molecular construction – was actually acquired. What appears to move is a few points of general answer hygiene, distributed almost arbitrarily across splits.

For the multitask work this practical leads into, that is still actionable, if in a more sober direction than we expected. It suggests the three families do not share enough procedural structure for gains to transfer implicitly, so a naive mixture should be expected to face interference rather than synergy. It also suggests that at 0.5B and 500 steps the more binding constraint may be capability rather than task allocation: before tuning a task mixture, it is worth establishing that any single task can be learned to a non-trivial level at all. Designs worth trying next are those that manage interference explicitly – balanced task sampling within a batch, per-task reward normalization so one family's reward scale cannot dominate the group advantage – evaluated on the full matrix at every checkpoint, on the benchmark rather than in-distribution.

The methodological findings may be the more durable contribution. A benchmark number is only as trustworthy as the control it is measured against. Our own held-out set told a confident, encouraging, and wrong story: it reported counting improving when the benchmark shows it degrading, and it scored a constant string as highly as any trained model on constraint generation. Three inexpensive checks caught this – a second, independent evaluation source; an answer-diversity statistic; and a constant-answer control. The last is worth stating as a general rule: *any reward defined by a verifier should be paired with a control that measures what the verifier accepts for free.* Without it, the model that looks best may simply be the one that has collapsed hardest, which is precisely what we observed.

References

[1] Mark Chen, Jerry Tworek, Heewoo Jun, et al. 2021. Evaluating Large Language Models Trained on Code. In *arXiv preprint arXiv:2107.03374*.

[2] Bradley Efron and Robert J. Tibshirani. 1993. *An Introduction to the Bootstrap*. Chapman & Hall/CRC.

[3] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, pp. 611–626.

[4] Greg Landrum. 2024. RDKit: Open-source Cheminformatics. https://www.rdkit.org. (2024).

```
./scripts/reproduce.sh datasets  # generate training questions (seed 42)
./scripts/reproduce.sh train     # single-task GRPO runs
./scripts/reproduce.sh eval      # full model x split matrix
./scripts/reproduce.sh report    # figures + summary tables
pytest -q                        # unit tests for the statistics
```

Listing 1: Full reproduction pipeline. Stages are idempotent and skip completed work.

[5] Quinn McNemar. 1947. Note on the Sampling Error of the Difference between Correlated Proportions or Percentages. *Psychometrika*, 12, 2, 153–157.

[6] Qwen Team. 2024. Qwen2.5 Technical Report. arXiv preprint arXiv:2412.15115. (2024).

[7] Philipp Seidl, Sebastian Sanokowski, Pieter-Jan Hoedt, and Günter Klambauer. 2025. MolecularIQ: A Benchmark for Verifiable Molecular Reasoning in Large Language Models. *arXiv preprint*. Benchmark and verifier: https://github.com/ml-jku/moleculariq-core.

[8] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. 2024. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. In *arXiv preprint arXiv:2402.03300*.

[9] Leandro von Werra, Younes Belkada, Lewis Tunstall, Edward Beeching, Tristan Thrush, Nathan Lambert, Shengyi Huang, Kashif Rasul, and Quentin Gallouédec. 2020. TRL: Transformer Reinforcement Learning. https://github.com/huggingface/trl. (2020).

[10] David Weininger. 1988. SMILES, a Chemical Language and Information System. 1. Introduction to Methodology and Encoding Rules. *Journal of Chemical Information and Computer Sciences*, 28, 1, 31–36.

[11] Edwin B. Wilson. 1927. Probable Inference, the Law of Succession, and Statistical Inference. *Journal of the American Statistical Association*, 22, 158, 209–212.

Appendix A. Reproducing These Results

Every number and figure in this report is produced by the accompanying repository. From a clean checkout with the pinned environment installed (`scripts/setup_leonardo.sh`):

Randomness is pinned throughout: dataset generation and training use seed 42, evaluation uses seed 0. On the pinned software stack and the same GPU model this reproduces the reported numbers exactly; across different GPU models, floating-point non-determinism in the attention kernels can move individual accuracies by a few tenths of a percent, well inside the reported confidence intervals.

Per-cell numbers, confidence intervals, p-values and answer-diversity statistics are written to `summary.csv` and `summary.md` in each group's plot directory

under `results/matrix/plots/`.